



VULNERABILIDADES STACK VS HEAP

ALUMNOS:
SAMUEL GARCÍA
NICOLÁS NEVÁREZ

QUE ES UN BUFFER OVERFLOW?

Un buffer overflow ocurre cuando un programa escribe más datos en un buffer (espacio de memoria) de los que puede contener.

Consecuencias:

- Corrupción de datos
- Fallos del programa (crashes)
- Vulnerabilidades de seguridad críticas
- Ejecución de código malicioso

En programación:

Cuando se desborda un buffer en memoria, los datos extra

sobrescriben:

- Otras variables del programa
- Direcciones de retorno
- Estructuras de control del sistema



TIPO DE MEM STACK VS HEAP

STACK (Memoria Estática):

- Gestión automática
- Variables locales de funciones
- Tamaño limitado
- Rápida asignación/liberación
- Variables contiguas en memoria

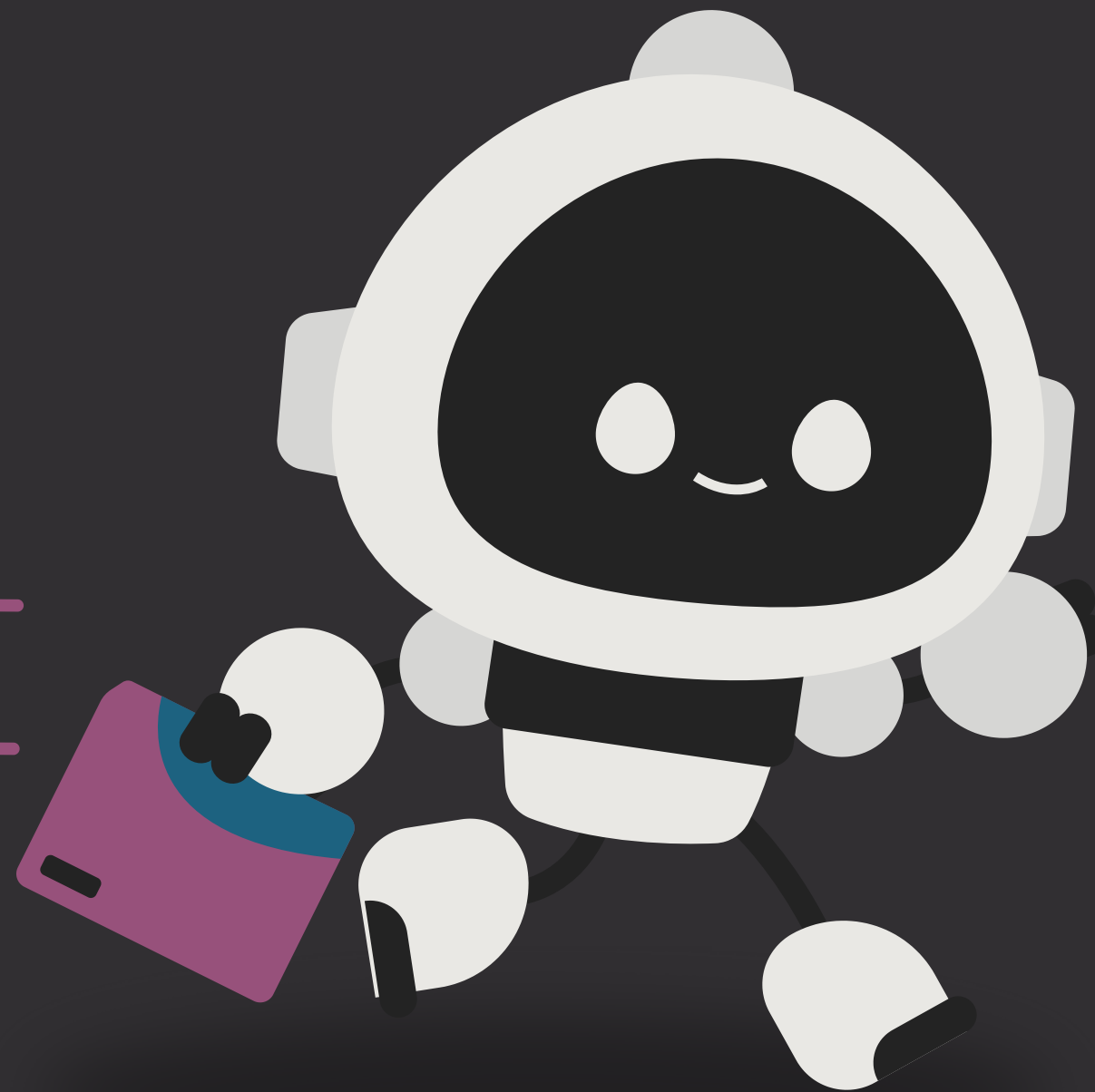
HEAP (Memoria Dinámica):

- Gestión manual
- Memoria solicitada explícitamente
- Tamaño limitado solo por RAM disponible
- Más lenta que stack
- Bloques no necesariamente contiguos

DIFERENCIAS CLAVE EN EL BUFFER OVERFLOW

En STACK:

Las variables están
juntas → overflow
afecta variables
vecinas
inmediatamente



En HEAP:

Los bloques están
separados →
overflow afecta
metadata del
sistema de
memoria

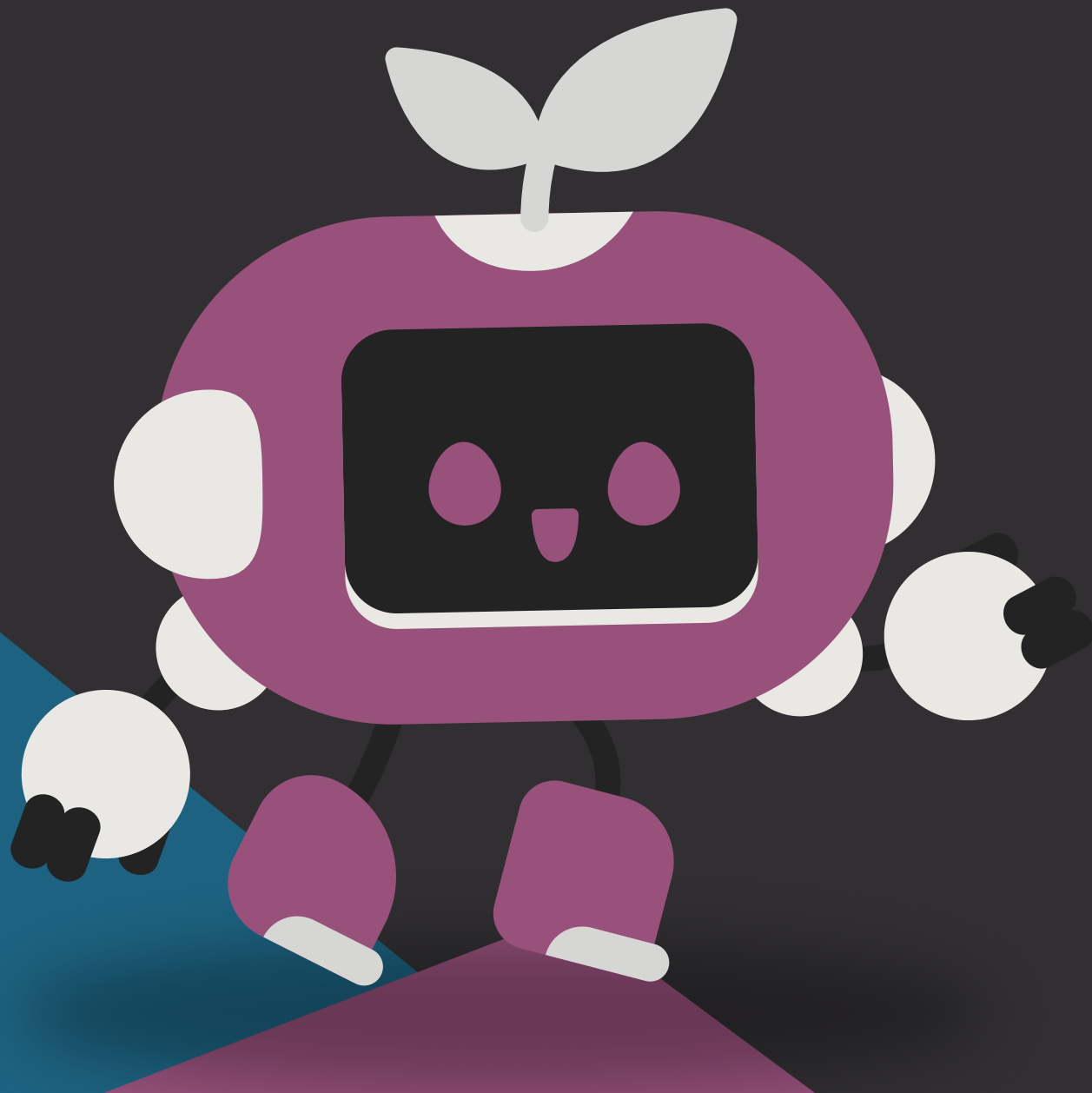
LA VULNERABILIDAD



STRCPY()

el objetivo de strcpy es copiar una cadena de caracteres desde un origen hasta su destino.

El problema viene cuando no verifica si tiene suficiente espacio y copia todo sobreponiendo el siguiente espacio hasta encontrar el carácter nulo.

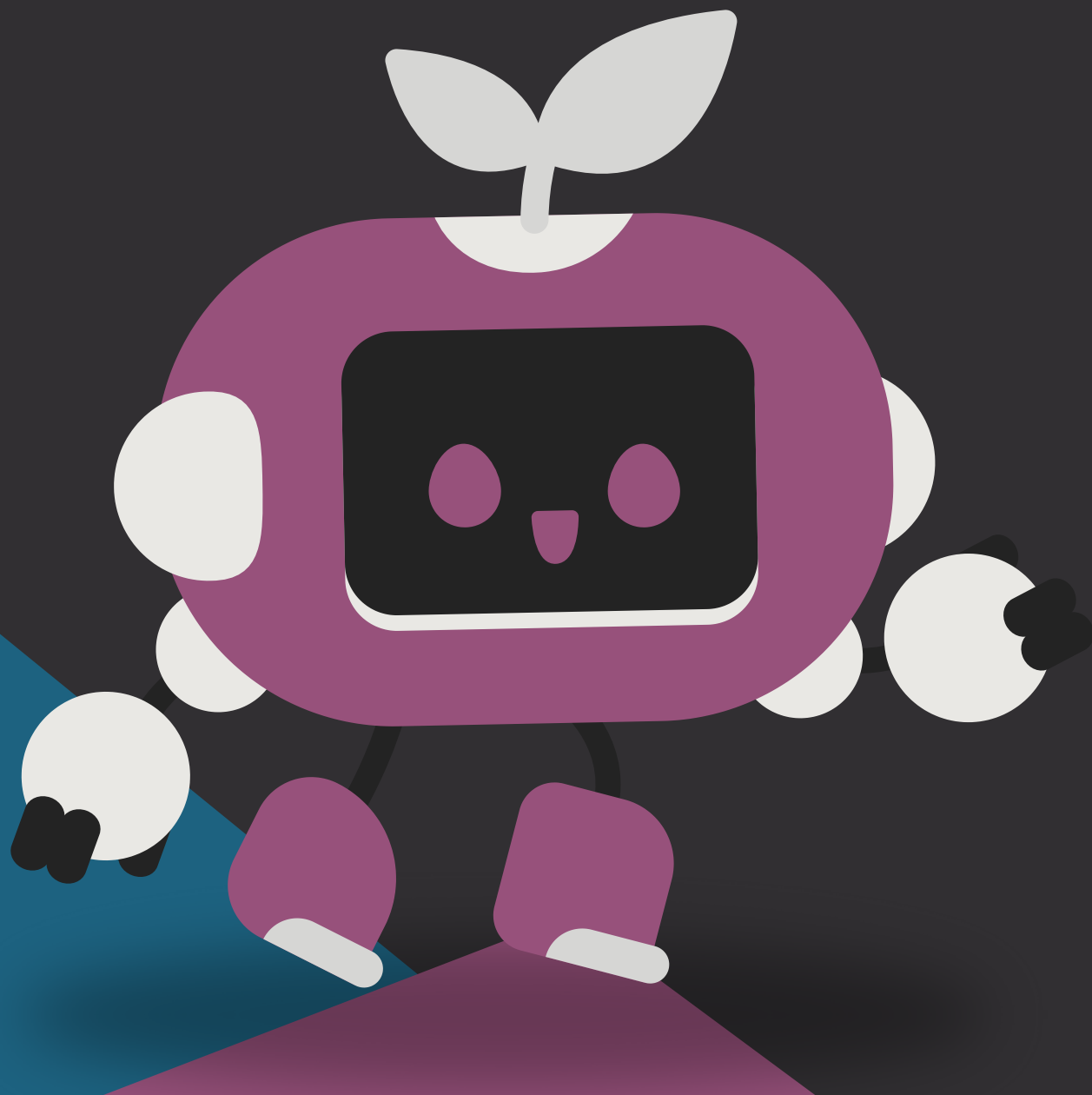


SU CURA

STRNCPY()

el objetivo de strncpy es copiar una cadena de caracteres hasta n desde el origen hasta su destino

Si verifica el límite de caracteres, jamás copiará más ni se sobrepondrá a otros espacios en memoria que no le pertenezcan.



DATOS CLAVE

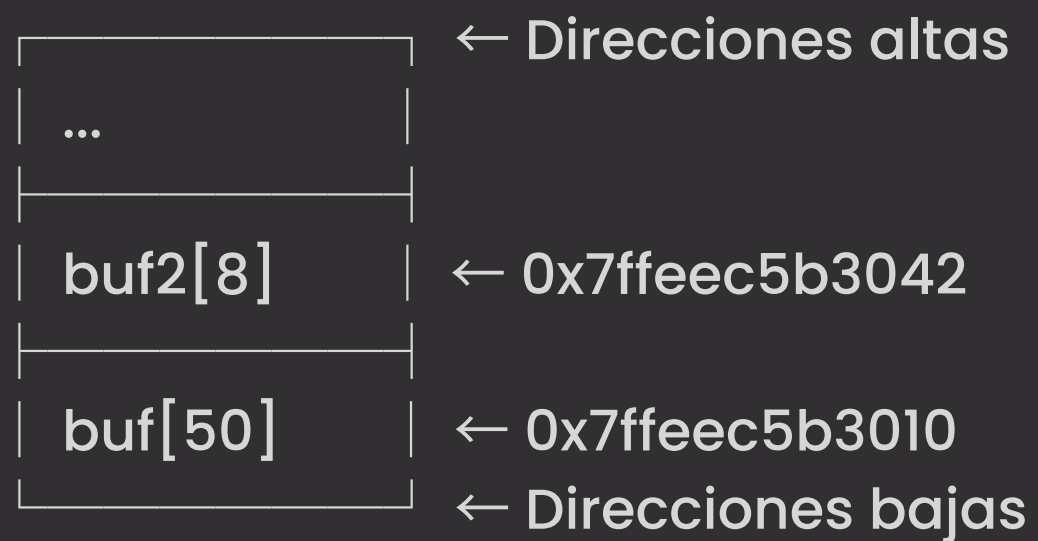
Usaremos lo siguiente:

- `sizeof(buf) - 1`
(Reservamos 1 byte para `'\0'`)
- `buf[49] = '\0'`
(Garantizamos el terminador nulo manualmente)

¿Por qué agregar `'\0'` manualmente?

Si la cadena origen es más larga que el límite, `strncpy` NO agrega el terminador nulo automáticamente. Debemos agregarlo manualmente para evitar comportamiento indefinido.

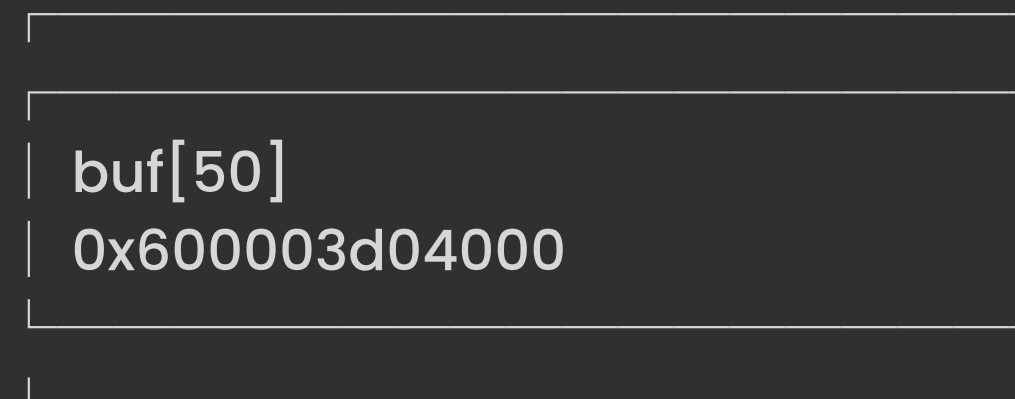
DIFERENCIA CRÍTICA



Diferencia: 50 bytes (contiguas)

Overflow en Stack: buf[50] se llena y continúa tanto escribiendo como sobrescribiendo en buf2 inmediatamente

HEAP:



STACK:

buf2[8]
0x16fdff257

Diferencia: Miles de bytes entre uno y otro

Overflow en Heap: buf se desborda y sobrescribe metadata del heap o bloques vecinos, buf2 está en stack, NO se afecta

◆◆CONCLUSIONES◆◆

- **El Buffer Overflow es Crítico:** Es un error común que afecta tanto al Stack (memoria estática) como al Heap (memoria dinámica), causando desde crashes hasta graves fallos de seguridad.
- **Stack vs. Heap:** La corrupción en el Stack suele ser inmediata y visible. En el Heap, puede ser "silenciosa" (más difícil de detectar) pero es igualmente peligrosa.
- **Las Protecciones NO son la Cura:** Los sistemas operativos modernos (como macOS) tienen protecciones que pueden ocultar el problema, pero NO lo eliminan. Un error "silencioso" es a menudo más peligroso.
- **La Prevención es Simple y Vital**



THANK
YOU

